

Arquitetura da API de Pedidos

Plataforma RESTful para gestão de pedidos de e-commerce, estruturada em Clean Architecture com CQRS, persistência poliglota e idempotência de compra.

PADRÃO ARQUITETURAL

STACK

CQRS · Clean Architecture

.NET 9 · SQL Server · MongoDB

1 Visão geral

API RESTful para gerenciamento de pedidos de e-commerce, construída em Clean Architecture com CQRS e persistência poliglota: **SQL Server** atua como banco de escrita (fonte da verdade, transacional) e **MongoDB** como banco de leitura (desnormalizado, otimizado para consulta). A criação de pedidos é idempotente, repetir a mesma requisição nunca duplica uma compra.

2 Domínio - status do pedido

O ciclo de vida de um pedido é modelado como uma máquina de estados explícita. Cada transição é um método protegido no agregado `Order` (`Process()`, `Ship()`, `Cancel()`); uma transição inválida lança uma exceção de domínio, mantendo os invariantes sempre consistentes.

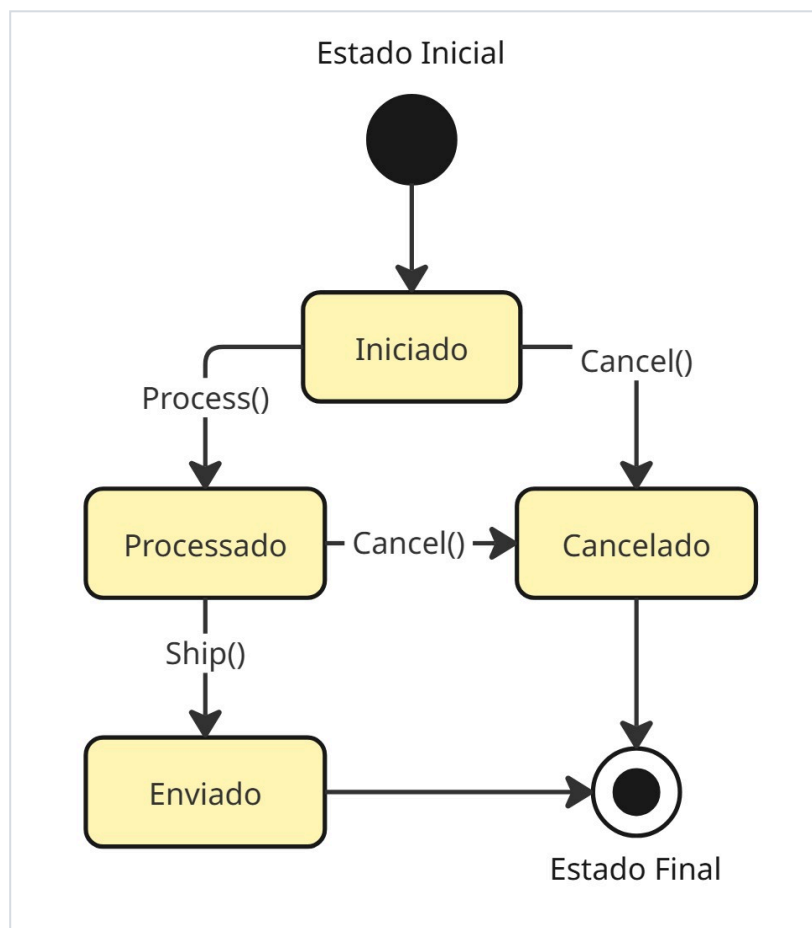


Figura 1. Transições de status permitidas para um pedido.

Transição	Permitida a partir de
Alterar pedido (PUT)	Iniciado
Processar	Iniciado
Enviar	Processado
Cancelar	Iniciado ou Processado

3 Arquitetura

A solução separa o caminho de escrita do caminho de leitura. Comandos percorrem o domínio e persistem no SQL Server; consultas leem o modelo desnormalizado no MongoDB. A sincronização entre os dois lados ocorre de forma assíncrona via padrão Outbox.

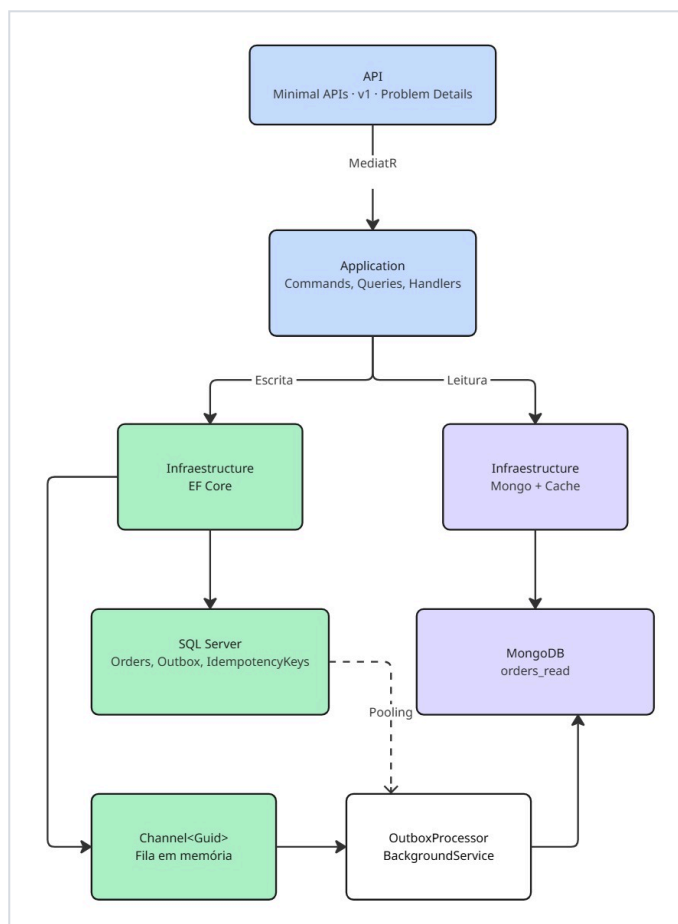


Figura 2. Camadas, caminho de escrita e caminho de leitura.

Camadas

- **Domain** - `Order`, `OrderItem`, `Buyer`, regras de invariante e transição. Sem dependências externas.
- **Application** - Commands e Queries via MediatR, com pipeline behaviors de idempotência, validação e logging.

- **Infrastructure** - EF Core (SQL Server), driver MongoDB, `OutboxProcessor` , `IMemoryCache` e Serilog.
- **Api** - Minimal APIs, OpenAPI nativo com Scalar, middleware de erros e health checks.

4 Fluxo de criação de pedido (idempotente)

1. Requisição `POST /api/v1/orders` com o header `Idempotency-Key` .
2. O pipeline verifica a key numa tabela dedicada: se já existir com o mesmo payload, retorna a resposta original sem recriar nada; a mesma key com payload diferente responde `409 Conflict` .
3. O handler valida os invariantes (comprador, status, ao menos um item, preço) e persiste `Order` , a mensagem de outbox e a idempotency key em uma única transação no SQL Server.
4. O commit sinaliza um canal em memória (`Channel<Guid>`) que acorda o `OutboxProcessor` , projetando o pedido no MongoDB em milissegundos. O polling periódico atua apenas como rede de segurança, por exemplo após um reinício do processo.

5 Endpoints

Método	Rota	Origem
POST	<code>/api/v1/orders</code>	Command → SQL Server
GET	<code>/api/v1/orders</code> , filtro por status/comprador, paginado	Query → MongoDB + cache
GET	<code>/api/v1/orders/{id}</code>	Query → MongoDB + cache
PUT	<code>/api/v1/orders/{id}</code> , somente status Iniciado	Command → SQL Server
POST	<code>/api/v1/orders/{id}/process</code> · <code>/ship</code> · <code>/cancel</code>	Command → SQL Server
DELETE	<code>/api/v1/orders/{id}</code> , soft-delete (cancela)	Command → SQL Server

6 Decisões-chave

Decisão	Escolha	Por quê
Mediator CQRS	MediatR	Pipeline behaviors prontos para idempotência, validação e logging
Persistência	SQL Server (escrita) + MongoDB (leitura)	CQRS real, não simbólico
Idempotência	Header <code>Idempotency-Key</code> + tabela dedicada, mesma transação do pedido	Garantia atômica contra duplicidade
Sincronização do read model	Outbox + <code>Channel<T></code> em memória (push) + polling de fallback	Baixa latência sem infraestrutura extra
Erros de domínio	Exceções específicas → Problem Details	Regras de negócio autoexplicativas na borda do agregado
Estilo de API	Minimal APIs com versionamento por URL	Simplicidade e explicitação das versões
Cache de leitura	<code>IMemoryCache</code> na listagem	Sem dependência de cache distribuído
Logging	Serilog estruturado	Observabilidade desde o início

7 Stack tecnológica

.NET 9 · EF Core · SQL Server · MongoDB · MediatR · Serilog · xUnit + FluentAssertions + Moq · Docker
Compose (Api + SQL Server + MongoDB)

8 Riscos principais

- **Consistência eventual** - a leitura pode, em teoria, atrasar em relação à escrita. Mitigado pelo sinal via `Channel<T>`, com latência na prática de milissegundos.
- **Mensagem de outbox presa** - se o MongoDB ficar indisponível, a mensagem permanece não processada e é reprocessada no próximo ciclo de polling; nenhuma mensagem é perdida.